Studio Nx

백엔드 230 — 디바이스 스키마

2026-10-18

Backend(FastAPI/Python) 230 — 디바이스 스키마 (악기 · 이펙트 · 연결선)

항목	내용
파일	`studionx/backend/app/schemas/device.py`
작업	S1-09 기본 악기·이펙트: 악기·이펙트·연결선 요청과 응답 (PRD 02 FR-245, FR-263, FR-267, FR-269)
작성	Claude (AI), 2026-10-18, 사용자 검토
이전 · 다음	229 플러그인 스키마 · 231 트랙 스키마

1. 하는 일

트랙에 붙는 악기(`track_instruments`), 이펙트(`track_plugins`), 연결선(`track_plugin_links`)의 요청과 응답 모양을 정합니다. 요청 6개, 응답 3개를 둡니다.

모델	하는 일
`TrackInstrumentCreateRequest`	악기 붙이기. `plugin_id` 필수, `slot_index`, `params`, `volume_db`, `pan`, `is_solo`, `is_enabled`
`TrackInstrumentUpdateRequest`	악기 고치기. 모든 칸이 선택입니다 (보낸 칸만 바뀝니다)
`TrackInstrumentResponse`	악기 한 칸의 응답
`TrackPluginCreateRequest`	이펙트 붙이기. `slot_index`(체인 순서), `node_x`, `node_y`, `params`, `is_bypassed`
`TrackPluginUpdateRequest`	이펙트 고치기 (순서 바꾸기, 카드 자리 옮기기, 파라미터, 바이패스)
`TrackPluginResponse`	이펙트 한 칸의 응답
`TrackPluginLinkCreateRequest`	연결선 만들기. `from`, `to`, 포트, `gain_db`
`TrackPluginLinkUpdateRequest`	연결선 고치기. 지금 API 에는 연결선 수정 경로가 없어 보내지 않습니다 (남겨 둔 여지만 씁니다)
`TrackPluginLinkResponse`	연결선 한 줄의 응답

주요 범위는 DB.md 를 그대로 따릅니다. 볼륨은 -60.00~+24.00 dB, 팬은 -1.00~1.00, 칸 번호와 포트는 0 이상입니다.

2. 언제 쓰나

쓰는 곳	
`POST /tracks/{id}/instruments` · `PATCH /tracks/{id}/instruments/{iid}` (239)	악기 붙이기·고치기 요청을 받습니다
`POST /tracks/{id}/plugins` · `PATCH /tracks/{id}/plugins/{pid}` (240)	이펙트 붙이기·고치기 요청을 받습니다
`POST /tracks/{id}/links` (241)	연결선 만들기 요청을 받습니다
`track_service` (233~236)	요청 값을 함수 인자로 넘기고 응답을 만듭니다

3. 코드

악기 요청 (붙이기 / 고치기)

```
class TrackInstrumentCreateRequest(BaseModel):
    plugin_id: int = Field(ge=1)
    slot_index: int = Field(ge=0, le=127, default=0)
    params: dict[str, Any] | None = None
    volume_db: Decimal = Field(default=Decimal("0.00"), ge=-60, le=24)
    pan: Decimal = Field(default=Decimal("0.00"), ge=-1, le=1)
    is_solo: bool = False
    is_enabled: bool = True


class TrackInstrumentUpdateRequest(BaseModel):
    slot_index: int | None = Field(default=None, ge=0, le=127)
    params: dict[str, Any] | None = None
    volume_db: Decimal | None = Field(default=None, ge=-60, le=24)
    pan: Decimal | None = Field(default=None, ge=-1, le=1)
    is_solo: bool | None = None
    is_enabled: bool | None = None
```

이펙트 요청 (붙이기)

```
class TrackPluginCreateRequest(BaseModel):
    plugin_id: int = Field(ge=1)
    slot_index: int = Field(ge=0, le=127, default=0)
    node_x: int | None = None
    node_y: int | None = None
    params: dict[str, Any] = Field(default_factory=dict)
    is_bypassed: bool = False
```

연결선 요청 (만들기)

```
class TrackPluginLinkCreateRequest(BaseModel):
    from_track_plugin_id: int | None = None
    from_port: int = Field(default=0, ge=0, le=15)
    to_track_plugin_id: int | None = None
    to_port: int = Field(default=0, ge=0, le=15)
    gain_db: Decimal = Field(default=Decimal("0.00"), ge=-60, le=24)
```

문법 풀이

식	뜻
`Field(ge=1)`	1 이상이어야 합니다. 없는 번호(0, 음수)를 미리 막아 422 로 답합니다
`Field(default=Decimal("0.00"), ge=-60, le=24)`	기본값과 함께 아래위 한계를 정합니다. 문자열 `"0.00"` 을 `Decimal` 로 바꿔 정확한 소수를 씁니다
`Field(default_factory=dict)`	매 요청마다 **새** 빈 사전을 만듭니다. `default={}` 로 쓰면 같은 사전을 함께 쓰는 위험이 있습니다
`int | None = None` (고치기 요청)	안 보낸 칸과 0 을 보낸 칸을 구분합니다. 라우터가 `exclude_unset=True` 로 안 보낸 칸을 걸러 냅니다

식	뜻
`Literal["CHAIN", "PATCHER"]`	라우팅 방식 두 값만 허용합니다 (231)

4. 왜 이렇게

선택	이유
붙이기와 고치기를 따로	붙일 때는 값이 다 있어야 하고(기본값 포함), 고칠 때는 보낸 칸만 바뀌어야 합니다. 한 모델로 두면 빈 값이 덮어쓸 수 있습니다
범위를 스키마에서 미리 막음	서비스와 DB 까지 가지 않고 422 로 바로 답합니다. DB CHECK 와 같은 한계를 화면에도 알려줍니다
`params` 를 `dict` 로	플러그인마다 파라미터가 다릅니다. 스키마를 플러그인마다 만들지 않고 사전으로 받습니다
`node_x`·`node_y` 를 빼도 되게	체인(CHAIN) 방식에서는 자리를 쓰지 않습니다. 갈래(PATCHER)로 바꿀 때만 넣습니다 (FR-269)
연결선에 `from`·`to` 를 비울 수 있게	비어 있으면 트랙 입력과 출력입니다. 한 표로 갈래 전체를 그릴 수 있습니다 (FR-267)
`TrackPluginLinkUpdateRequest` 를 남겨 둠	지금은 선을 지우고 다시 긋지만, 나중에 선만 옮기는 화면이 생기면 이 모델을 씁니다

5. 시험

`tests/test_track_routes.py` 가 악기·이펙트·연결선의 붙이기, 고치기, 떼기와 함께 다음을 확인합니다.

- 붙이기 응답의 칸 이름(`INSTRUMENT_KEYS`, `EFFECT_KEYS`, `LINK_KEYS`)과 기본값(`is_enabled` = true, `is_bypassed` = false)
- 악기 자리에 이펙트를 넣으면 400, 이펙트 자리에 악기를 넣으면 400
- 범위를 벗어난 값(색 형식, 볼륨 99.00, 칸 번호)은 422
- 같은 칸에 두 번 넣거나 남이 쓰는 칸으로 옮기면 400